DPTO. ELECTRICIDAD Y ELECTRÓNICA

TPMMEE - 2021/22

OVERFLOW DE NÚMEROS ENTEROS EN ARDUINO

AUTOR: Lucas Crespo

24/03/2022

Hoja de Guarda

TABLA DE CONTENIDOS

1. Objetivos 4

2. Almacenamiento en memoria de Arduino 4

3. Forzando un overflow con números enteros 5

3.1 Números enteros: Int 5

3.2 Números enteros sin signo: Unsigned Int 8

PRÁCTICA DE DISEÑO DE CIRCUITOS COMBINACIONALES

TPMMEE - 2021/22

TPMMEE - 2021/22

ARDUINO OVERFLOW

——— Lucas Crespo

1. Objetivos.

Se desea comparar el comportamiento de los tipos de variable int y unsigned int al rebasar el máximo de valores que pueden almacenar.

2. Almacenamiento en memoria de Arduino.

Arduino es uno de los productos de la compañía con mismo nombre que consiste en una placas de desarrollo de hardware para construir dispositivos digitales y dispositivos interactivos para aplicaciones en el mundo real.

El dispositivo que vamos a emplear en esta práctica, Arduino UNO, posee una memoria interna de 16 bits, otorgando a la placa la capacidad de registrar hasta $2^{16} = 65536$ cifras.

Uno de los tipos de datos más comunes a almacenar son los números enteros, o números de *tipo int.* Los valores que este tipo de datos abarca incluyen números positivos y negativos. Como el límite de memoria es de 65536, los valores int irán desde -32768 hasta 32767, con el 0 incluido.

Se puede indicar a arduino que no se empleen valores negativos al declarar el tipo int con la función unsigned el cual, como su nombre dice, quita el signo de la cifra. Por lo tanto, los enteros de *tipo unsigned int* podrán tomar valores de 0 a 65535 cifras.

3. Forzando un overflow con números enteros.

Creamos un programa en el que inicializamos una variable tipo entero, y analizamos qué ocurre si la aumentamos desde 0 de manera indefinida.

3.1. Números enteros: Int.

Este programa inicializa la variable tipo entero solo con *int*. Al ejecutar el programa en un bucle infinito, aumentando el valor de variable en cada pasada, se puede observar que al llegar al máximo predicho (32767), el siguiente valor es el mínimo posible: -32768.

Tras eso, el contador sigue incrementando la cifra de uno en uno en cada pasada de manera normal.

```
// INICIALIZACIÓN A 0 DE VARIABLE CONTADOR
int contador;

// INICIALIZACIÓN DE CONDICIONES INICIALES DEL PROGRAMA
void setup() {

  // Iniciar la comunicación entre arduino y ordenador a
  // una velocidad de 9600 bits por segundo.
  Serial.begin(9600);

}

// LOOP INFINITO; PROGRAMA PRINCIPAL
void loop() {
  // Imprimimos en la consola, con salto de línea,
  // la variable contador aumenetándola una vez cada pasada
  // contador ++ ---> contador = contador + 1
  Serial.println (contador++);

}
```

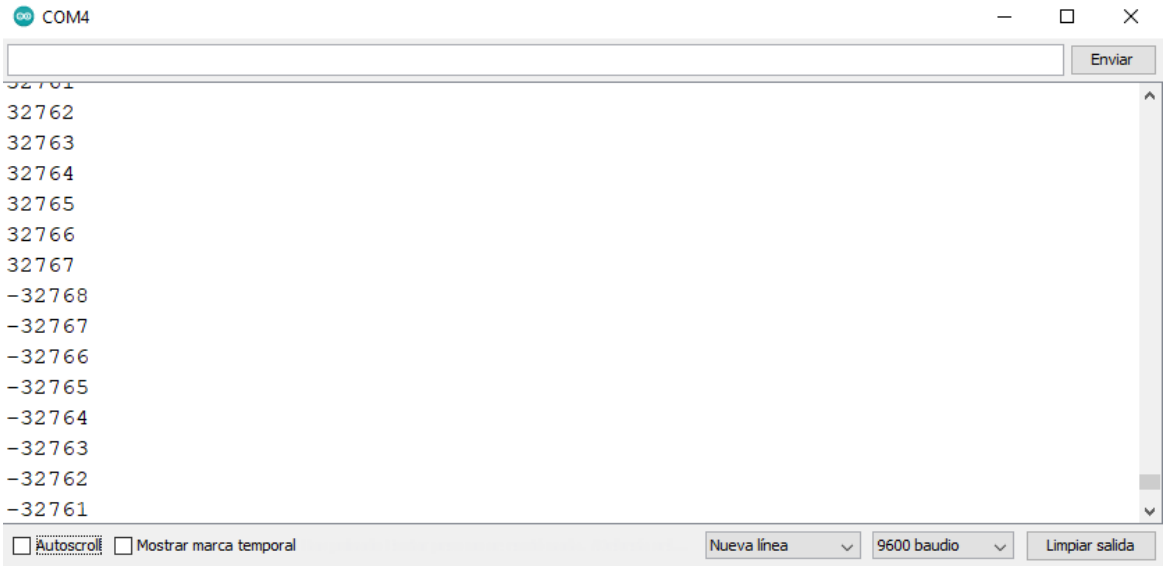

Figura 1: Arriba: Programa contador de enteros.
Debajo: Salida por terminal cuando ocurre la saturación

El cambio de máximo a mínimo ocurre porque el arduino está representando números negativos en complemento a 2 (Ca2). Este método de representación consiste en invertir el número binario cambiando los ceros por unos y, a continuación, sumando un bit.

Por ejemplo, 32767 en binario es 11111111. Si queremos hacer este número negativo con Ca2 primero cambiamos ceros por unos y unos por ceros: 00000000; y sumamos un bit: 00000001. De esta manera - 32767 en Ca2 es 00000001.

TIPO INT			
BITS DE MEMORIA (8)		REPRESENTACIÓN	EVENTO
Binario	Digital		
00000000	0	Binario	Primer número positivo
11111111	32767	Binario	Último número positivo
1 | 00000000	32768	Binario	Intento de representar el siguiente positivo: Overflow. NO se muestra este número
00000000	- 32768	Ca2	Como no se puede meter un bit más, el resto de números posibles han quedado a 0. Como el rango es de - 32768 a 32767, el primer número a representar es negativo. Usando Ca2 para representar números negativos, queda que - 32768 es el primero.
00000001	- 32767	Ca2	Continúa la cuenta normal en Ca2 hasta que se lleguen a los números positivos.

3.2. Números enteros sin signo: Unsigned Int.

Este programa inicializa la variable tipo entero solo con *int*. Al ejecutar el programa en un bucle infinito, aumentando el valor de variable en cada pasada, se puede observar que al llegar al máximo predicho (65535), el siguiente valor es el mínimo posible: 0.

Tras eso, el contador sigue incrementando la cifra de uno en uno en cada pasada de manera normal.

```
// INICIALIZACIÓN A 0 DE VARIABLE CONTADOR
unsigned int contador;

// INICIALIZACIÓN DE CONDICIONES INICIALES DEL PROGRAMA
void setup() {

  // Iniciar la comunicación entre arduino y ordenador a
  // una velocidad de 9600 bits por segundo.
  Serial.begin(9600);

}

// LOOP INFINITO; PROGRAMA PRINCIPAL
void loop() {
  // Imprimimos en la consola, con salto de línea,
  // la variable contador aumenetándola una vez cada pasada
  // contador ++ ---> contador = contador + 1
  Serial.println (contador++);

}
```

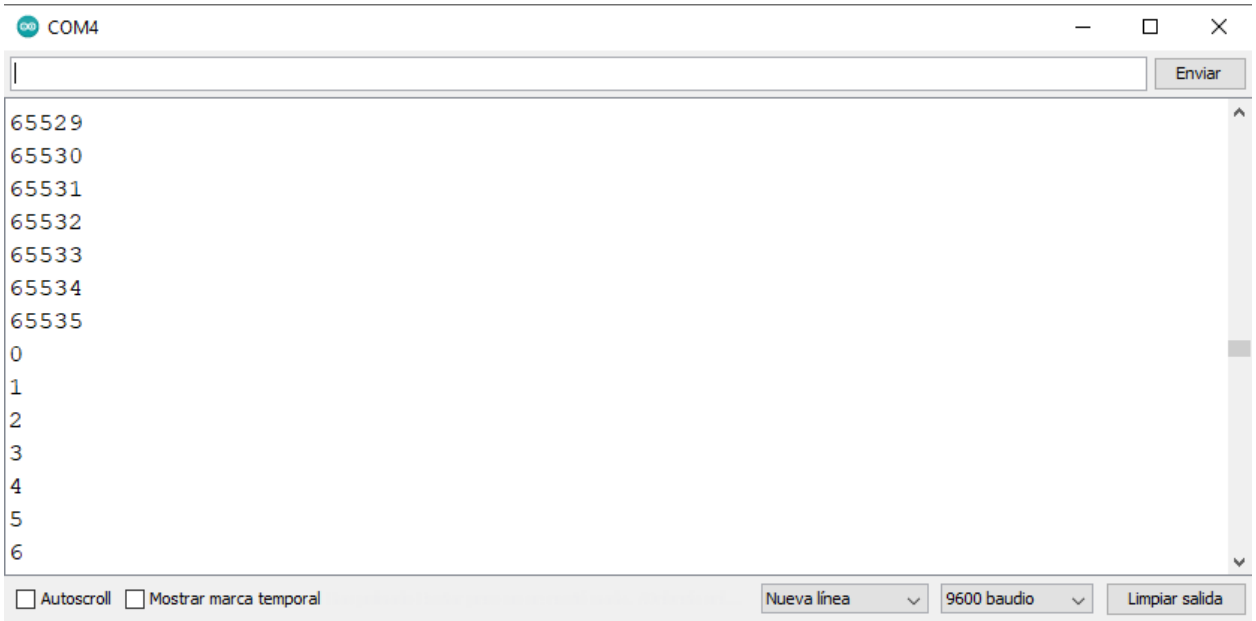

Figura 2: Arriba: Programa contador de enteros sin signo.
Debajo: Salida por terminal cuando ocurre la saturación

El cambio de máximo a mínimo es más fácil de entender en este caso, ya que no hay números negativos que representar. De esta manera, la memoria simplemente se queda sin espacio para añadir más bits y la cuenta regresa a 0 como se muestra a continuación:

TIPO UNSIGNED INT			
BITS DE MEMORIA (16)		REPRESENTACIÓN	EVENTO
Binario	Digital		
0000000000000000	0	Binario	Primer número positivo
1111111111111111	65535	Binario	Último número positivo
1 | 0000000000000000	65535	Binario	Intento de representar el siguiente positivo: Overflow. NO se muestra este número
0000000000000000	0	Binario	Como no se puede meter un bit más, el resto de números posibles han quedado a 0. Como el rango es de 0 a 65535, el primer número a representar es positivo: 0.
0000000000000000	1	Binario	Continúa la cuenta normal

Hoja de Guarda